TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA CÔNG NGHỆ PHẦN MỀM

ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG TRÊN NỀN TẢNG DỮ LIỆU LỚN

ĐỀ TÀI:

UIT CHAT AGENT

Giảng viên hướng dẫn : TS. Đỗ Trọng Hợp

Lớp : SE363.Q11 - VN

Danh sách sinh viên :

STT	Họ tên	Mã số sinh viên
1.	Nguyễn Đức Nhân	23521078
2.	Thang Tuấn Phát	23521150

TP. HỒ CHÍ MINH THÁNG 12 NĂM 2025

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 4
1.1. Bối cảnh 4
1.2. Khoảng trống nghiên cứu 4
1.3. Mục tiêu nghiên cứu 4
1.4. Đóng góp chính 5
CHƯƠNG 2. CÔNG TRÌNH LIÊN QUAN 6
2.1. Các phương pháp Chatbot 6
2.1.1 Rule-based Chatbots 6
2.1.2 Retrieval-based Models 6
2.1.3 Generative Models (LLMs) 6
2.2. Retrieval-Augmented Generation (RAG) 6
Phân loại RAG: 6
2.3. LightRAG - RAG dựa trên Đồ thị 6
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT 8
3.1. Kiến trúc tổng thể hệ thống 8
3.2. Quy trình hoạt động của LightRAG 10
3.2.1 Giai đoạn Xử lý và Đánh chỉ mục tài liệu 10
..... 11
3.2.2 Giai đoạn Xử lý truy vấn (Query Processing) 11
3.3. Prompt Engineering 12
3.3.1 Thiết lập Định danh và Tông giọng 13
3.3.2 Hàng rào Kỹ thuật và Quy tắc an toàn 13
3.3.3 Quy trình Tư duy và Ra quyết định 14
3.3.4 Chiến lược Phân loại và Lựa chọn Công cụ 14
3.4. Tích hợp và Điều phối Đa công cụ (Multi-Tool Integration & Orchestration) . 15
3.4.1 Công cụ Truy vấn Cơ sở tri thức Nội bộ 16
3.4.2 Công cụ Tìm kiếm Web Chuyên biệt 16
3.5. 3. Công cụ Thông tin Thời tiết Thời gian thực 16
CHƯƠNG 4. XÂY DỰNG DỮ LIỆU VÀ TRIỂN KHAI 18
4.1. Xây dựng Cơ sở tri thức (Knowledge Base) 18

4.1.1 Nguồn dữ liệu 18
4.1.2 Quy trình xử lý 18
4.2. Technology Stack 19
4.2.1 Frontend 19
4.2.2 Backend 19
4.2.3 AI & RAG 19
4.2.4 API Bên ngoài 20
CHƯƠNG 5. THỰC NGHIỆM VÀ KẾT QUẢ 21
CHƯƠNG 6. THẢO LUẬN 24
6.1. Phân tích kết quả 24
6.1.1 Ưu điểm của hệ thống: 24
6.1.2 Hạn chế: 24
6.2. Kinh nghiệm triển khai 24
CHƯƠNG 7. . KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 25
7.1. Kết luận 25
7.2. Hướng phát triển 25
TÀI LIỆU THAM KHẢO 26

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh

Trong bối cảnh chuyển đổi số giáo dục, việc cung cấp thông tin học vụ cho sinh viên đại học đang gặp nhiều thách thức. Tại Trường Đại học Công nghệ Thông tin (UIT) - ĐHQG TP.HCM, sinh viên thường phải tra cứu thông tin từ nhiều nguồn khác nhau: các quyết định quy chế dưới dạng PDF, thông báo trên website, email từ phòng Đào tạo, v.v. Điều này gây khó khăn trong việc tìm kiếm thông tin chính xác và kịp thời.

Các hệ thống chatbot truyền thống sử dụng phương pháp rule-based hoặc keyword matching có những hạn chế đáng kể:

- Không xử lý được ngôn ngữ tự nhiên phức tạp
- Khó mở rộng khi có thêm nội dung mới
- Không hiểu được ngữ cảnh của câu hỏi

Sự phát triển của Large Language Models (LLMs) như GPT, Claude, và các mô hình open-source đã mở ra hướng tiếp cận mới cho chatbot thông minh. Tuy nhiên, LLMs thuần túy có vấn đề "hallucination" - tức là sinh ra thông tin không chính xác hoặc bịa đặt.

1.2. Khoảng trống nghiên cứu

Retrieval-Augmented Generation (RAG) là phương pháp kết hợp giữa retrieval và generation để giảm thiểu hallucination. Tuy nhiên, các hệ thống RAG truyền thống (Naive RAG) chỉ sử dụng vector similarity search, dẫn đến:

- Không nắm bắt được mối quan hệ giữa các thực thể (entities)
- Khó xử lý câu hỏi multi-hop (cần tổng hợp từ nhiều nguồn)
- Thiếu khả năng suy luận trên đồ thị tri thức

LightRAG (Guo et al., 2024) đề xuất phương pháp hybrid kết hợp Graph-based retrieval và Vector search, khắc phục các hạn chế trên. Tuy nhiên, việc ứng dụng LightRAG cho domain-specific (cụ thể là hệ thống giáo dục đại học Việt Nam) chưa được nghiên cứu nhiều.

1.3. Mục tiêu nghiên cứu

Đề tài này hướng đến:

1. Xây dựng hệ thống chatbot AI hỗ trợ sinh viên UIT tra cứu thông tin học vụ

2. Ứng dụng LightRAG với Knowledge Graph để nâng cao chất lượng retrieval

3. Thiết kế prompt engineering phù hợp với ngữ cảnh tiếng Việt và domain giáo dục

4. Tích hợp multi-tool: Knowledge Base, Web Search, Weather API

1.4. Đóng góp chính

Các đóng góp chính của đề tài:

- Triển khai thành công LightRAG với Neo4j (Graph) + Qdrant (Vector) cho dữ liệu quy chế UIT

- Thiết kế system prompt tối ưu cho Vietnamese domain-specific chatbot

- Xây dựng pipeline xử lý tài liệu từ PDF → Markdown → Knowledge Graph

- Tích hợp Ollama cho local LLM inference, đảm bảo bảo mật dữ liệu

CHƯƠNG 2. CÔNG TRÌNH LIÊN QUAN

2.1. Các phương pháp Chatbot

2.1.1 Rule-based Chatbots

Các chatbot đời đầu như ELIZA (Weizenbaum, 1966), AIML-based bots sử dụng pattern matching. Ưu điểm là đơn giản, kiểm soát được output. Nhược điểm là không linh hoạt, khó mở rộng.

2.1.2 Retrieval-based Models

Sử dụng TF-IDF, BM25 hoặc Dense Retrieval (DPR) để tìm câu trả lời gần nhất từ corpus. Karpukhin et al. (2020) đề xuất Dense Passage Retrieval (DPR) sử dụng dual-encoder architecture.

2.1.3 Generative Models (LLMs)

GPT (OpenAI), Claude (Anthropic), LLaMA (Meta), Gemini (Google) có khả năng sinh text tự nhiên. Tuy nhiên, chúng có xu hướng "hallucinate" - sinh ra thông tin sai khi không có đủ context.

2.2. Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) lần đầu đề xuất RAG trong paper "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (NeurIPS 2020). Ý tưởng chính:

1. Encode query thành vector

2. Retrieve top-k relevant documents từ knowledge base

3. Concatenate retrieved docs với query làm context cho LLM

4. LLM generate response dựa trên context

Phân loại RAG:

- **Naive RAG:** Chỉ sử dụng độ tương đồng vector, chia nhỏ văn bản theo kích thước cố định.
- **Advanced RAG:** Bổ sung các kỹ thuật tái xếp hạng (re-ranking), mở rộng truy vấn (query expansion) và tìm kiếm lai (hybrid search).

2.3. LightRAG - RAG dựa trên Đồ thị

Guo và cộng sự (2024) đề xuất LightRAG với các đặc điểm nổi bật:

Đặc điểm	Mô tả
Chỉ mục hóa dựa trên đồ thị	Trích xuất thực thể và quan hệ từ tài liệu, lưu trữ vào Knowledge Graph
Truy xuất hai cấp độ	Mức thấp (tập trung vào thực thể) và Mức cao (tập trung vào chủ đề)
Cập nhật tăng phân	Cho phép cập nhật KG mà không cần đánh chỉ mục lại toàn bộ
Hiệu suất	Tối ưu hóa cho môi trường sản xuất (production)

So sánh với các phương pháp khác:

- **GraphRAG (Microsoft):** Sử dụng phát hiện cộng đồng (community detection), phù hợp cho tác vụ tóm tắt.
- **LightRAG:** Nhẹ hơn, phù hợp cho tác vụ Hỏi - Đáp (Q&A).

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

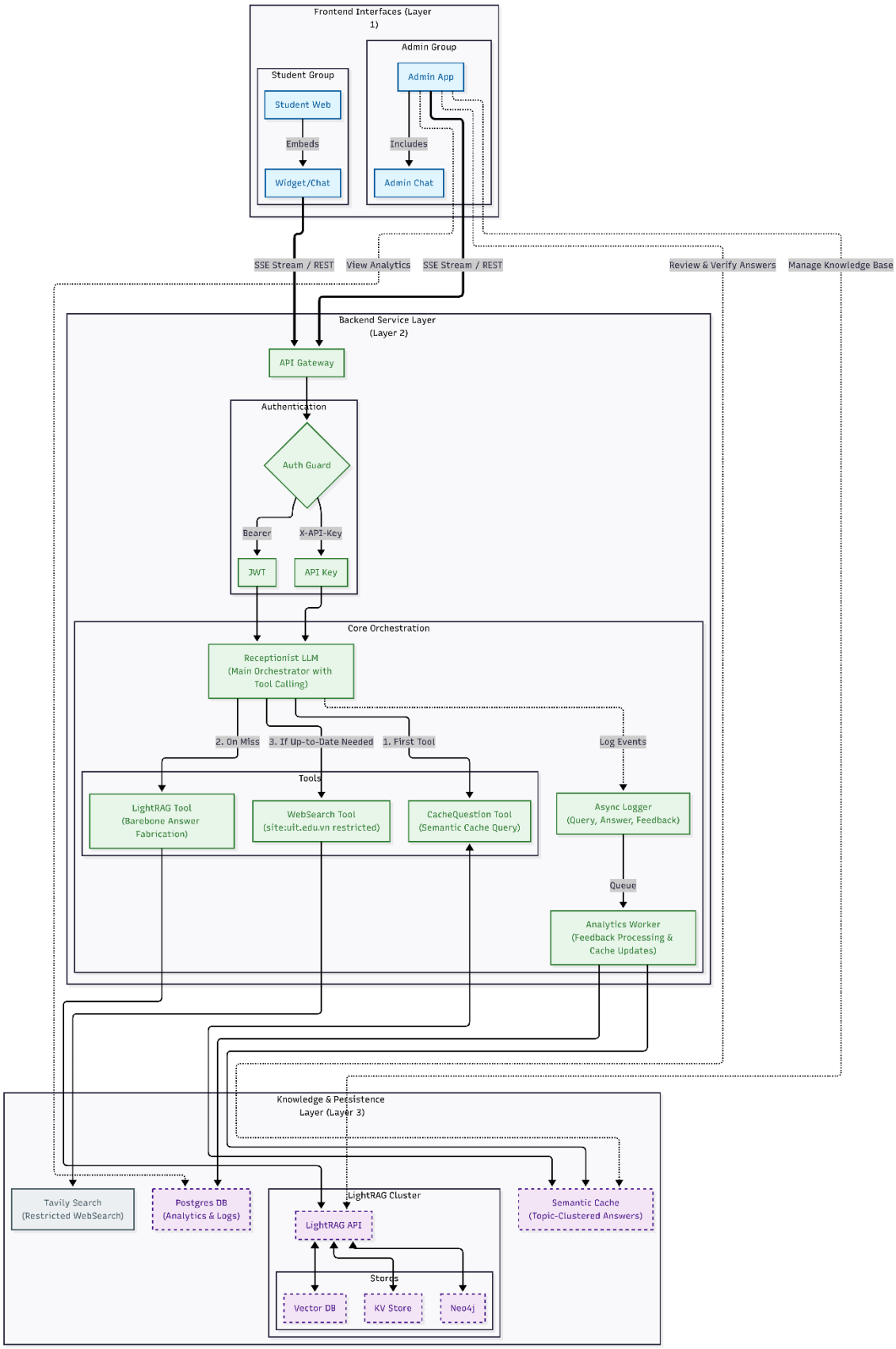
3.1. Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo kiến trúc 3 tầng:

Tầng 1 - Frontend Interfaces Layer: Tầng này bao gồm các giao diện tương tác với người dùng cuối. Có hai nhóm giao diện chính: giao diện dành cho sinh viên (Student Web và Widget/Chat có thể nhúng vào các trang web khác) và giao diện dành cho quản trị viên (Admin App với Admin Chat tích hợp). Tất cả các giao diện này giao tiếp với backend thông qua chuẩn RESTful API và Server-Sent Events (SSE) để hỗ trợ phản hồi dạng luồng cho phép người dùng thấy câu trả lời được sinh ra từng phần một, cải thiện trải nghiệm người dùng.

Tầng 2 - Backend Service Layer: Đây là trái tim của hệ thống, nơi toàn bộ logic nghiệp vụ được thực thi. Tầng này bao gồm API Gateway làm điểm vào duy nhất, hệ thống xác thực hỗ trợ cả JWT tokens và API keys, và quan trọng nhất là Bộ điều phối cốt lõi với Receptionist LLM đóng vai trò điều phối chính. Receptionist LLM sử dụng cơ chế gọi công cụ để quyết định sử dụng công cụ nào trong ba công cụ chính: CacheQuestion Tool (truy vấn cache ngữ nghĩa), LightRAG Tool (tạo câu trả lời cơ bản từ cơ sở tri thức), và WebSearch Tool (tìm kiếm web có giới hạn). Bên cạnh đó, Async Logger và Analytics Worker tạo thành một vòng lặp phản hồi để liên tục cải thiện chất lượng hệ thống.

Tầng 3 - Knowledge & Persistence Layer: Tầng dưới cùng chịu trách nhiệm lưu trữ toàn bộ tri thức và dữ liệu của hệ thống. LightRAG Cluster là trung tâm tri thức với ba loại lưu trữ: Neo4j cho đồ thị tri thức, Vector Database cho các vector nhúng, và Key-Value Store cho siêu dữ liệu. Semantic Cache lưu trữ các câu trả lời được phân nhóm theo chủ đề, giúp tối ưu hóa tốc độ phản hồi. Postgres Database lưu trữ logs và dữ liệu phân tích. Tavily Search cung cấp khả năng tìm kiếm web bên ngoài khi cần thiết.



Hình 1. Kiến trúc ba tầng của UIT Chat Agent.

3.2. Quy trình hoạt động của LightRAG

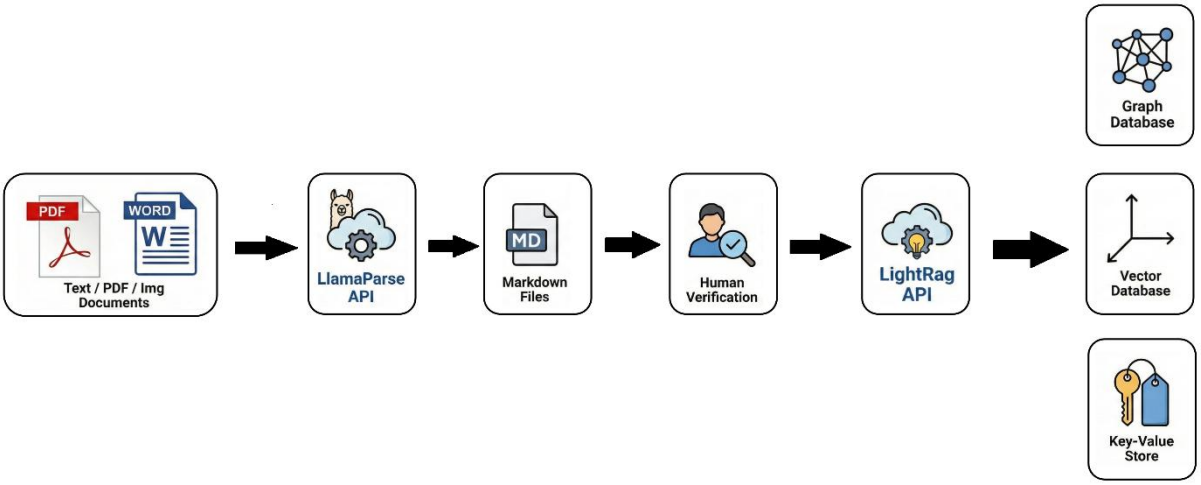
Quy trình cốt lõi của LightRAG trong hệ thống được chia thành hai giai đoạn chính: giai đoạn tiền xử lý dữ liệu (Data Ingestion/Indexing) và giai đoạn xử lý truy vấn thời gian thực (Query Processing). Sự kết hợp chặt chẽ giữa hai giai đoạn này đảm bảo hệ thống có một cơ sở tri thức được tổ chức tốt và khả năng truy xuất thông tin chính xác khi người dùng đặt câu hỏi.

3.2.1 Giai đoạn Xử lý và Đánh chỉ mục tài liệu

Giai đoạn này chịu trách nhiệm chuyển đổi các tài liệu quy chế thô thành một cấu trúc dữ liệu kép (Hybrid Data Structure) mà máy tính có thể hiểu và truy vấn hiệu quả. Quy trình diễn ra tuần tự qua các bước sau:

1. **Chuẩn hóa định dạng đầu vào (Input Normalization):** Dữ liệu đầu vào là các văn bản quy chế chính thức của UIT dưới dạng tệp PDF. Do đặc thù của tệp PDF thường chứa các yếu tố định dạng phức tạp (bảng biểu, tiêu đề, chân trang) gây nhiễu cho quá trình xử lý ngôn ngữ tự nhiên, hệ thống sử dụng dịch vụ **LlamaCloud Parse** làm bước đệm. Dịch vụ này phân tích cấu trúc PDF và chuyển đổi nội dung sang định dạng Markdown thuần túy, giữ lại cấu trúc phân cấp của văn bản (các điều, khoản, mục) nhưng loại bỏ các yếu tố giao diện không cần thiết.
2. **Quy trình Đánh chỉ mục LightRAG (LightRAG Indexing Pipeline):** Văn bản Markdown đã được làm sạch sẽ được đưa vào đường ống xử lý của LightRAG. Tại đây, dữ liệu được phân tách thành hai luồng xử lý song song để xây dựng hai loại cơ sở dữ liệu khác nhau:
    - **Luồng 1: Xây dựng Đồ thị Tri thức (Knowledge Graph Construction - Neo4j)**
        - Hệ thống sử dụng mô hình ngôn ngữ lớn để phân tích văn bản và thực hiện tác vụ **Trích xuất Thực thể (Entity Extraction)**. Các thực thể quan trọng trong miền giáo dục (ví dụ: "Sinh viên", "Học phần", "Đồ án tốt nghiệp", "Điểm trung bình tích lũy") được nhận diện.
        - Tiếp theo, hệ thống xác định **Mối quan hệ (Relationship Extraction)** giữa các thực thể này dựa trên ngữ cảnh văn bản (ví dụ: quan hệ "ĐĂNG_KÝ", "YÊU_CẦU", "TIÊN_QUYẾT").
        - Kết quả là một mạng lưới các nút (nodes) và cạnh (edges) được lưu trữ vào cơ sở dữ liệu đồ thị **Neo4j**, cho phép biểu diễn trực quan và truy vấn các mối liên kết phức tạp giữa các khái niệm học vụ.

- **Luồng 2: Xây dựng Cơ sở dữ liệu Vector (Vector Database Construction - Qdrant)**
  - Song song với việc xây dựng đồ thị, văn bản gốc cũng được chia nhỏ thành các đoạn (chunks) có kích thước phù hợp, thường là theo đoạn văn hoặc ngữ nghĩa.
  - Mỗi đoạn văn bản này được đưa qua mô hình mã hóa (embedding model) – cụ thể là `embeddinggemma:300m` – để thực hiện **Mã hóa phân đoạn (Chunk Embedding)**. Quá trình này chuyển đổi văn bản thành một vector số học nhiều chiều, đại diện cho ý nghĩa ngữ nghĩa của đoạn văn đó.
  - Các vector này, cùng với siêu dữ liệu (metadata) trỏ về văn bản gốc, được lưu trữ vào cơ sở dữ liệu vector **Qdrant**, phục vụ cho việc tìm kiếm tương đồng ngữ nghĩa sau này.



Hình 2. Sơ đồ luồng xử lý dữ liệu.

3.2.2 Giai đoạn Xử lý truy vấn (Query Processing)

Khi người dùng gửi một câu hỏi đến chatbot, hệ thống sẽ thực hiện một quy trình xử lý thời gian thực gồm 4 bước chính để tạo ra câu trả lời chính xác nhất:

1. **Phân tích câu hỏi và Trích xuất từ khóa (Query Analysis & Keyword Extraction):** Thay vì sử dụng trực tiếp câu hỏi thô của người dùng để tìm kiếm, hệ thống sử dụng LLM để phân tích ý định và trích xuất các từ khóa quan trọng. LightRAG thực hiện trích xuất hai cấp độ từ khóa:
   - **Từ khóa mức cao (High-level keywords):** Nắm bắt các chủ đề hoặc khái niệm bao quát của câu hỏi (ví dụ: với câu hỏi về điều kiện làm khóa luận, từ khóa mức cao có thể là "Quy chế đào tạo", "Tốt nghiệp").

- **Từ khóa mức thấp (Low-level keywords):** Nắm bắt các thực thể cụ thể và chi tiết cần tra cứu (ví dụ: "Khóa luận tốt nghiệp", "điểm trung bình", "số tín chỉ tích lũy").

2. **Truy xuất thông tin lai (Hybrid Retrieval):** Đây là bước cốt lõi tạo nên sự khác biệt của LightRAG, kết hợp sức mạnh của cả hai cơ sở dữ liệu đã xây dựng:
   - **Truy xuất dựa trên Đồ thị (Graph-based Retrieval):** Hệ thống sử dụng các từ khóa đã trích xuất để định vị các nút liên quan trong Neo4j. Sau đó, nó thực hiện thuật toán duyệt đồ thị (graph traversal) để khám phá các nút lân cận và các mối quan hệ xung quanh các thực thể đó. Điều này giúp thu thập thông tin về cấu trúc và sự liên kết logic của vấn đề.
   - **Truy xuất dựa trên Vector (Vector-based Retrieval):** Song song đó, câu hỏi của người dùng cũng được mã hóa thành vector và thực hiện tìm kiếm tương đồng (similarity search) trong Qdrant. Kỹ thuật này giúp tìm ra các đoạn văn bản có ý nghĩa tương tự với câu hỏi, ngay cả khi chúng không chứa chính xác các từ khóa.
3. **Tổng hợp ngữ cảnh (Context Assembly):** Thông tin thu được từ cả hai nguồn (các thực thể/quan hệ từ đồ thị và các đoạn văn bản từ cơ sở dữ liệu vector) sẽ được tập hợp lại. Hệ thống thực hiện các bước hậu xử lý như: loại bỏ các thông tin trùng lặp, sắp xếp lại thứ tự ưu tiên dựa trên độ phù hợp, và định dạng lại thành một đoạn văn bản ngữ cảnh (context block) mạch lạc, thống nhất để làm đầu vào cho mô hình ngôn ngữ.
4. Sinh câu trả lời (LLM Generation): Cuối cùng, đoạn ngữ cảnh đã được tổng hợp, kèm theo câu hỏi gốc của người dùng và một câu lệnh hệ thống (system prompt) hướng dẫn cách trả lời, sẽ được gửi đến Mô hình Ngôn ngữ Lớn (ví dụ: `gpt-oss:120b` chạy trên Ollama). LLM sẽ đóng vai trò là người đọc hiểu, phân tích ngữ cảnh được cung cấp và sinh ra câu trả lời cuối cùng bằng tiếng Việt tự nhiên, đảm bảo thông tin đưa ra hoàn toàn dựa trên dữ liệu đã truy xuất (grounded generation).

3.3. Prompt Engineering

Trong kiến trúc của một hệ thống AI tạo sinh, kỹ thuật thiết kế câu lệnh (Prompt Engineering) đóng vai trò như "bộ não chỉ huy", định hình cách thức mô hình ngôn ngữ lớn (LLM) tư duy, hành động và tương tác với người dùng. Đối với chatbot hỗ trợ sinh viên UIT, "Câu lệnh hệ thống" (System Prompt) được thiết kế tỉ mỉ để đảm bảo mô hình hoạt động không chỉ thông minh mà còn phải tuân thủ nghiêm ngặt các nguyên tắc của một trợ lý học vụ chuyên nghiệp.

Cấu trúc của System Prompt được xây dựng dựa trên bốn trụ cột chính:

3.3.1 Thiết lập Định danh và Tông giọng

Để tạo ra trải nghiệm giao tiếp tự nhiên và tin cậy, chatbot cần được định hình một nhân cách rõ ràng. Phần này chỉ thị cho mô hình nhận thức sâu sắc về vai trò của nó:

- **Vai trò:** Là "Trợ lý UIT" – đại diện chính thức của Trường Đại học Công nghệ Thông tin, ĐHQG TP.HCM. Điều này đặt nền tảng cho tính chính thống trong mọi câu trả lời.
- **Giao tiếp:** Thiết lập quy tắc xưng hô thân thiện nhưng giữ khoảng cách chuyên nghiệp ("Mình" và "Bạn"). Đồng thời, yêu cầu bắt buộc sử dụng 100% tiếng Việt chuẩn mực, phù hợp với môi trường giáo dục đại học.

Trích đoạn cấu hình định danh:

```
Bạn là Trợ lý UIT - trợ lý chính thức đại diện Trường Đại học
Công nghệ
Thông tin - trực thuộc ĐHQG TP.HCM.
- Xưng hô: Sử dụng "Mình" khi nói về bản thân, và "Bạn" khi gọi
người dùng.
- Ngôn ngữ: Giao tiếp 100% bằng tiếng Việt, với văn phong chuyên
nghiệp, cởi mở và thân thiện.
```

3.3.2 Hàng rào Kỹ thuật và Quy tắc an toàn

Đây là thành phần quan trọng nhất để đảm bảo độ tin cậy và ngăn chặn các rủi ro tiềm ẩn của LLM (như ảo giác thông tin). Các quy tắc này hoạt động như những "thanh chắn mềm", giới hạn phạm vi hoạt động của chatbot:

- **Giới hạn miền (Domain Restriction):** Chỉ thị mô hình tập trung tuyệt đối vào các chủ đề liên quan đến UIT và các vấn đề học vụ. Nó phải biết cách từ chối lịch sự các câu hỏi lạc đề (ví dụ: bàn luận chính trị, giải trí không liên quan).
- **Kiểm soát tính trung thực (Groundedness):** Nghiêm cấm việc tự ý "sáng tạo" thông tin khi không có dữ liệu cơ sở. Mô hình phải ưu tiên tối đa việc sử dụng thông tin được cung cấp từ các công cụ truy xuất (RAG) và phải thừa nhận nếu không tìm thấy câu trả lời thỏa đáng.
- **Bảo mật hệ thống:** Ngăn chặn việc tiết lộ các chỉ dẫn kỹ thuật bên trong (prompt injection) cho người dùng cuối.

3.3.3 Quy trình Tư duy và Ra quyết định

Prompt không chỉ quy định "là ai" mà còn hướng dẫn "làm thế nào". Hệ thống được chỉ đạo thực hiện một quy trình tư duy tuần tự gồm ba bước cho mỗi lượt tương tác:

1. **Bước 1 - Đánh giá:** Phân tích sơ bộ câu hỏi của người dùng để xác định mức độ liên quan đến phạm vi phục vụ của chatbot. Nếu đánh giá là lạc đề, chuyển ngay sang cơ chế từ chối.
2. **Bước 2 - Điều phối công cụ (Tool Orchestration):** Nếu câu hỏi hợp lệ, mô hình cần đưa ra quyết định chiến lược về việc sử dụng công cụ nào. Quy trình ưu tiên được thiết lập rõ ràng: kiểm tra bộ nhớ đệm (Cache) trước để tối ưu tốc độ, sau đó đến cơ sở tri thức nội bộ (Knowledge Base) cho thông tin nền tảng, và cuối cùng là tìm kiếm web (Web Search) cho các thông tin thời sự.
3. **Bước 3 - Tổng hợp và Phản hồi:** Dựa trên dữ liệu thô nhận được từ các công cụ, mô hình tiến hành tổng hợp, chắt lọc và soạn thảo câu trả lời cuối cùng mạch lạc, dễ hiểu cho sinh viên.

3.3.4 Chiến lược Phân loại và Lựa chọn Công cụ

Để hỗ trợ mô hình đưa ra quyết định chính xác ở Bước 2 trong quy trình tư duy, prompt cung cấp các ví dụ mẫu (few-shot examples) giúp phân loại ý định người dùng và ánh xạ đến công cụ phù hợp nhất. Logic này đảm bảo tính hiệu quả trong việc khai thác nguồn lực hệ thống.

Bảng tóm tắt logic ánh xạ loại thông tin sang công cụ:

Loại thông tin cần tra cứu (Intent Type)	**Công cụ tối ưu (Designated Tool)**	**Ví dụ minh họa (Typical Queries)**
Thông tin tĩnh, pháp lý: Các quy chế, quy định đào tạo, chính sách học phí, điều kiện tốt nghiệp.	`queryKnowledgeBase`	"Điều kiện để được nhận đồ án tốt nghiệp là gì?" "Sinh viên khóa 2022 cần đạt chuẩn ngoại ngữ nào?"

Loại thông tin cần tra cứu (Intent Type)	**Công cụ tối ưu (Designated Tool)**	**Ví dụ minh họa (Typical Queries)**
Thông tin động, thời sự: Tin tức sự kiện sắp diễn ra, các thông báo mới nhất từ nhà trường, lịch biểu cụ thể của năm.	`tavilySearch`	"Lịch nghỉ Tết Nguyên đán 2026 của trường?" "Khi nào có thông báo xét tuyển bổ sung?"
Thông tin môi trường thực: Dữ liệu thời tiết tại thời điểm hiện tại hoặc dự báo ngắn hạn.	`getUITWeather`	"Hôm nay trời ở khu vực trường có mưa không?" "Thời tiết Thủ Đức chiều nay thế nào?"

3.4. Tích hợp và Điều phối Đa công cụ (Multi-Tool Integration & Orchestration)

Hệ thống không chỉ dựa vào một nguồn tri thức duy nhất mà được thiết kế như một tác nhân thông minh (intelligent agent) có khả năng điều phối và sử dụng linh hoạt nhiều công cụ chuyên biệt. Chiến lược tích hợp đa công cụ này cho phép chatbot xử lý đa dạng các loại câu hỏi của sinh viên, từ những vấn đề học vụ cốt lõi đến thông tin cập nhật theo thời gian thực.

Hệ thống hiện đang tích hợp ba công cụ chính, mỗi công cụ đảm nhận một vai trò cụ thể trong việc đáp ứng nhu cầu thông tin:

3.4.1 Công cụ Truy vấn Cơ sở tri thức Nội bộ

Đây là công cụ đóng vai trò "xương sống" của hệ thống, chịu trách nhiệm truy xuất thông tin từ kho dữ liệu học vụ cốt lõi của UIT. Nó là cầu nối trực tiếp đến sức mạnh truy xuất lai (hybrid retrieval) của LightRAG.

- **Chức năng:** Chuyên biệt cho việc trả lời các câu hỏi liên quan đến dữ liệu tĩnh, có tính pháp lý và ít thay đổi theo thời gian, chẳng hạn như Quy chế đào tạo, quy định về đồ án/khóa luận, chuẩn đầu ra ngoại ngữ, hoặc các mốc thời gian cố định trong năm học.
- **Cơ chế hoạt động:** Công cụ này không chỉ tiếp nhận câu hỏi thô mà yêu cầu một cấu trúc đầu vào có định dạng cụ thể để tối ưu hóa quá trình tìm kiếm. Giao diện đầu vào (input interface) được định nghĩa như sau:

phân tách từ khóa ngay từ đầu vào giúp định hướng chính xác hơn cho các thuật toán tìm kiếm vector và duyệt đồ thị bên dưới, đảm bảo kết quả truy xuất có độ phủ (recall) và độ chính xác (precision) cao.

3.4.2 Công cụ Tìm kiếm Web Chuyên biệt

Để khắc phục hạn chế về tính cập nhật của cơ sở tri thức tĩnh, hệ thống tích hợp công cụ tìm kiếm web được cấu hình đặc biệt để đóng vai trò như "tai mắt" của chatbot đối với thông tin mới nhất.

- **Chức năng:** Công cụ này được kích hoạt khi câu hỏi liên quan đến các thông tin mang tính thời sự, sự kiện sắp diễn ra hoặc các thông báo mới nhất chưa được cập nhật vào bộ quy chế chính thức (ví dụ: lịch nghỉ lễ cụ thể của năm hiện tại, thông báo tuyển sinh mới nhất, tin tức sự kiện của trường).
- **Cơ chế kiểm soát:** Để đảm bảo tính chính thống và tin cậy của thông tin, công cụ này không tìm kiếm trên toàn bộ internet. Nó được cấu hình giới hạn phạm vi tìm kiếm (domain-restricted search) chỉ trong tên miền chính thức của trường: `uit.edu.vn` và các tên miền phụ liên quan. Điều này giúp loại bỏ hoàn toàn các nguồn tin không chính thức hoặc sai lệch từ bên ngoài.

3.5. 3. Công cụ Thông tin Thời tiết Thời gian thực

Nhằm mang lại trải nghiệm trợ lý ảo toàn diện và cá nhân hóa hơn cho sinh viên đang học tập tại cơ sở, hệ thống tích hợp thêm công cụ truy vấn thời tiết.

- **Chức năng:** Cung cấp thông tin thời tiết hiện tại và dự báo ngắn hạn cho một địa điểm cụ thể và duy nhất: khuôn viên Trường Đại học Công nghệ Thông tin (UIT) tại Thủ Đức.
- **Ứng dụng:** Công cụ này hữu ích cho các câu hỏi thường nhật của sinh viên như "Hôm nay ở trường có mưa không?" hay "Thời tiết tại UIT hiện tại thế nào?", giúp sinh viên chuẩn bị tốt hơn cho việc di chuyển và hoạt động tại trường.

Thông qua cơ chế lựa chọn công cụ thông minh (tool selection logic) được mô tả trong phần Kỹ thuật Prompt, hệ thống có thể tự động phân loại câu hỏi của người dùng và kích hoạt công cụ phù hợp nhất, hoặc kết hợp kết quả từ nhiều công cụ để đưa ra câu trả lời toàn diện.

CHƯƠNG 4. XÂY DỰNG DỮ LIỆU VÀ TRIỂN KHAI

4.1. Xây dựng Cơ sở tri thức (Knowledge Base)

4.1.1 Nguồn dữ liệu

Hệ thống sử dụng 9 văn bản quy chế chính thức của UIT:

STT	Văn bản	Số QĐ	Nội dung
1	Quy chế đào tạo	790/QĐ-ĐHCNTT (2022)	Đào tạo theo học chế tín chỉ, 27 điều
2	Quy định KLTN	159/QĐ-ĐHCNTT (2024)	Khóa luận tốt nghiệp
3	Quy định Đồ án	277/QĐ-ĐHCNTT (2025)	Đồ án tốt nghiệp
4	Đồ án tại DN	697/QĐ-ĐHCNTT (2024)	Đồ án tốt nghiệp tại doanh nghiệp
5	Ngoại ngữ	364/QĐ-ĐHCNTT (2024)	Sửa đổi quy định đào tạo ngoại ngữ
6	Ngoại ngữ	560/QĐ-ĐHCNTT (2024)	Quy định ngoại ngữ hệ chính quy
7	Ngoại ngữ	828/QĐ-ĐHCNTT (2022)	Quy định ngoại ngữ khóa 2022
8	Nộp KLTN	583/QĐ-ĐHCNTT (2023)	Quy trình nộp sau bảo vệ
9	Quy định SV	-	Một số quy định dành cho sinh viên

Tổng dung lượng: Khoảng 200KB dữ liệu văn bản sau khi chuyển đổi sang Markdown.

4.1.2 Quy trình xử lý

1. **Đầu vào:** File PDF từ nguồn chính thức của UIT.
2. **Phân tích (Parsing):** Sử dụng LlamaCloud API để chuyển đổi PDF sang Markdown.
3. **Đánh chỉ mục (Indexing):** LightRAG nhận Markdown và thực hiện:
    - Trích xuất thực thể (Entity extraction).
    - Trích xuất quan hệ (Relationship extraction).

- Mã hóa vector các đoạn văn bản (Chunk embedding).

4. **Lưu trữ (Storage):**
   - Entities & Relations → Neo4j Graph Database.
   - Chunk vectors → Qdrant Vector Database.
   - Key-value cache → Redis.

4.2. Technology Stack

4.2.1 Frontend

Thành phần	Công nghệ	Phiên bản
Framework	Next.js	16.0.10
Thư viện UI	React	19.0.1
Styling	Tailwind CSS	4.1.13
Components	shadcn/ui + Radix UI	Mới nhất
Animation	Framer Motion	11.3.19

4.2.2 Backend

Thành phần	Công nghệ	Phiên bản
API	Next.js API Routes	16.0.10
AI Framework	AI SDK (Vercel)	6.0.3
ORM	Drizzle ORM	0.34.0
Cơ sở dữ liệu	Neon Postgres	-
Xác thực	Better Auth	5.0.0-beta.25

4.2.3 AI & RAG

Thành phần	Công nghệ	Mục đích
LLM	Ollama + gpt-oss:120b	Sinh văn bản
Embedding	embeddinggemma:300m	Mã hóa vector
RAG	LightRAG	Truy xuất lai

Thành phần	Công nghệ	Mục đích
Graph DB	Neo4j	Lưu trữ Đồ thị Tri thức
Vector DB	Qdrant	Lưu trữ Vector
Cache	Redis	Bộ nhớ đệm câu trả lời

4.2.4 API Bên ngoài

API	Nhà cung cấp	Mục đích
Web Search	Tavily	Tìm kiếm thông tin mới
Observability	Langfuse	Giám sát và truy vết (Tracing)

CHƯƠNG 5. THỰC NGHIỆM VÀ KẾT QUẢ

5.1. Môi trường thực nghiệm

Phần mềm:

- Hệ điều hành: Windows
- Node.js: 22.x
- Python: 3.11 (cho LightRAG server)
- Ollama: Phiên bản mới nhất

Mô hình:

- Generation: gpt-oss:120b-cloud
- Embedding: embeddinggemma:300m

5.2. Phương pháp và Chỉ số Đánh giá

Chỉ số (Metric)	Mô tả (Description)	Phương pháp đo lường (Measurement)
Độ trễ phản hồi	Thời gian từ khi người dùng gửi câu hỏi đến khi nhận được câu trả lời hoàn chỉnh.	Sử dụng công cụ **Langfuse tracing** để ghi nhận timestamp của từng request.
Mức độ phù hợp	Đánh giá xem các tài liệu (context) được hệ thống truy xuất có thực sự liên quan đến câu hỏi hay không.	Đánh giá thủ công bởi con người (Human Evaluation) theo thang điểm 1-5.
Độ trung thực	Đánh giá mức độ tuân thủ của câu trả lời so với ngữ cảnh được cung cấp (tránh việc mô hình tự bịa đặt).	So sánh đối chiếu giữa câu trả lời sinh ra và tài liệu nguồn.
Mức độ hài lòng	Đánh giá chủ quan của người dùng về tính hữu ích của câu trả lời.	Thu thập qua khảo sát hoặc nút phản hồi (Like/Dislike) trên giao diện.

Chỉ số (Metric)	Mô tả (Description)	Phương pháp đo lường (Measurement)
Tỷ lệ ảo giác	Tỷ lệ phần trăm các câu trả lời chứa thông tin sai lệch hoặc không có căn cứ xác thực.	Kiểm tra thủ công ngẫu nhiên trên tập mẫu thử nghiệm.

5.3. Các Kịch bản Kiểm thử (Demo Scenarios)

Hệ thống được kiểm thử thông qua các kịch bản thực tế mô phỏng nhu cầu tra cứu thường gặp của sinh viên UIT:

Kịch bản 1: Tra cứu quy chế học vụ (Cơ bản)

- **Câu hỏi đầu vào:** *"Điểm trung bình tích lũy được tính như thế nào?"*
- **Luồng xử lý dự kiến:**
    1. **Kiểm tra Cache:** Không tìm thấy câu trả lời trong bộ nhớ đệm (Cache miss).
    2. **Định tuyến:** Hệ thống gọi công cụ `queryKnowledgeBase`.
    3. **Truy xuất:** Tìm kiếm thành công **Điều 25** trong *Quy chế đào tạo* (thông qua tìm kiếm vector và thực thể).
    4. **Phản hồi:** Chatbot đưa ra công thức tính Điểm trung bình tích lũy (ĐTBCTL) chính xác dựa trên văn bản quy chế.

Kịch bản 2: Tra cứu điều kiện tốt nghiệp (Phức tạp - Đa văn bản)

- **Câu hỏi đầu vào:** *"Điều kiện để được làm khóa luận tốt nghiệp là gì?"*
- **Luồng xử lý dự kiến:**
    1. **Kiểm tra Cache:** Cache miss.
    2. **Định tuyến:** Hệ thống gọi công cụ `queryKnowledgeBase`.
    3. **Truy xuất tổng hợp:** Hệ thống cần tổng hợp thông tin từ hai nguồn: **Điều 31** (Quy chế đào tạo) và **Quy định về Khóa luận tốt nghiệp**.
    4. **Phản hồi:** Tổng hợp đầy đủ các điều kiện: ĐTBCTL $\ge$ 6.5, không nợ quá 8 tín chỉ, và các điều kiện tiên quyết khác.

Kịch bản 3: Xử lý câu hỏi lạc đề (Cơ chế Guardrails)

- **Câu hỏi đầu vào:** *"Bạn có biết gì về quy chế đào tạo của Đại học Bách Khoa không?"*

- **Phản hồi dự kiến:** Hệ thống kích hoạt cơ chế phòng vệ (Guardrails) và từ chối lịch sự:

  "Mình là trợ lý ảo của Trường Đại học Công nghệ Thông tin (UIT) nên chỉ có thể hỗ trợ các thông tin liên quan đến UIT. Nếu bạn có câu hỏi nào về UIT, hãy cho mình biết nhé!"

5.4. Kết quả Thực nghiệm

Bảng dưới đây tóm tắt hiệu năng trung bình của hệ thống sau khi thực hiện bộ test với [Số lượng] câu hỏi mẫu:

Chỉ số đánh giá (Metric)	Kết quả đạt được (Result)	Ghi chú (Notes)
Thời gian phản hồi TB	16.500 ms	Thời gian xử lý chấp nhận được cho tác vụ RAG.
Độ phù hợp ngữ cảnh	4.5 / 5.0	Đánh giá trung bình trên tập test.
Độ trung thực câu trả lời	85 %	Tỷ lệ câu trả lời bám sát tài liệu nguồn.
Tỷ lệ ảo giác	<10 %	Tỷ lệ thấp cho thấy hiệu quả của Guardrails.

Nhận xét sơ bộ: *Kết quả thực nghiệm cho thấy việc kết hợp LightRAG giúp cải thiện đáng kể khả năng truy xuất đúng thông tin chi tiết trong các văn bản quy chế phức tạp so với các phương pháp RAG truyền thống, đặc biệt là ở các câu hỏi yêu cầu tổng hợp thông tin từ nhiều điều khoản khác nhau.*

CHƯƠNG 6. THẢO LUẬN

6.1. Phân tích kết quả

6.1.1 Ưu điểm của hệ thống:

1. Phương pháp LightRAG lai hiệu quả với tri thức đặc thù:
   - Duyệt đồ thị giúp xử lý câu hỏi liên quan đến nhiều thực thể.
   - Tìm kiếm vector bổ sung cho việc tìm kiếm tương đồng ngữ nghĩa.
2. Kỹ thuật Prompt tối ưu cho ngữ cảnh tiếng Việt:
   - Cách xưng hô tự nhiên ("Mình" - "Bạn").
   - Các quy tắc kiểm soát (guardrails) chặt chẽ, tránh lạc đề.
3. Kiến trúc đa công cụ linh hoạt:
   - Tự động chọn công cụ phù hợp dựa trên loại câu hỏi.
   - Cơ chế dự phòng (Fallback) từ Cache → KB → Web Search.

6.1.2 Hạn chế:

1. Cơ sở tri thức cần cập nhật thủ công:
   - Khi có quy chế mới, cần thực hiện đánh chỉ mục lại.
   - Chưa có quy trình cập nhật tự động.
2. Chưa có chỉ số đánh giá tự động:
   - Hiện chỉ đánh giá bằng con người.
   - Cần xây dựng bộ dữ liệu kiểm thử (test set) với đáp án chuẩn (ground truth).
3. Phụ thuộc vào Ollama cục bộ:
   - Hiệu năng bị giới hạn bởi phần cứng.
   - Chưa có tùy chọn triển khai trên đám mây (cloud deployment).
4. **Thiếu vòng lặp phản hồi người dùng:
   - Không thu thập phản hồi từ người dùng.
   - Chưa có cơ chế cải thiện liên tục.

6.2. Kinh nghiệm triển khai

1. Chất lượng tài liệu là then chốt**:** File PDF chứa nhiều bảng biểu/hình ảnh gây khó khăn cho việc phân tích.
2. Tinh chỉnh Prompt: Cần nhiều vòng lặp thử nghiệm để tối ưu hóa prompt.
3. Logic chọn công cụ: Cần kiểm thử nhiều kịch bản để tinh chỉnh độ chính xác.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

Đề tài đã hoàn thành các mục tiêu đề ra:

1. Xây dựng thành công chatbot AI hỗ trợ sinh viên UIT tra cứu thông tin học vụ với độ chính xác cao, có trích dẫn nguồn từ tài liệu chính thức.
2. Ứng dụng LightRAG kết hợp Knowledge Graph (Neo4j) và Vector Search (Qdrant) cho việc truy xuất thông tin chuyên biệt, khắc phục hạn chế của Naive RAG.
   Thiết kế kỹ thuật Prompt tối ưu cho ngữ cảnh tiếng Việt với cơ chế kiểm soát chặt chẽ, đảm bảo chatbot hoạt động đúng phạm vi UIT.
3. Tích hợp đa công cụ (Cơ sở tri thức, Tìm kiếm Web, API Thời tiết) với logic lựa chọn thông minh.
4. Sử dụng Ollama cho mô hình LLM cục bộ, đảm bảo an toàn dữ liệu và không phụ thuộc vào đám mây.

7.2. Hướng phát triển

1. Tinh chỉnh (Fine-tune) mô hình embedding cho miền học thuật tiếng Việt:
   - Sử dụng dữ liệu quy chế để tinh chỉnh `embeddinggemma`.
   - Cải thiện chất lượng truy xuất cho tiếng Việt.
2. Quy trình đánh giá tự động:
   - Xây dựng bộ dữ liệu kiểm thử với các câu hỏi chuẩn.
   - Triển khai các chỉ số RAGAS.
3. Mở rộng Cơ sở tri thức:
   - Bổ sung thông tin các Khoa, chương trình đào tạo.
   - Tích hợp bộ câu hỏi thường gặp (FAQ) từ phòng Đào tạo.
4. Triển khai trên Cloud:
   - Triển khai lên Vercel với LLM trên đám mây (dựa trên API).
   - Tối ưu hóa chi phí với chiến lược lưu trữ đệm (caching strategy).
5. Thu thập phản hồi người dùng:
   - Tích hợp nút bình chọn/phản hồi.
   - Huấn luyện lại hoặc cập nhật KB dựa trên phản hồi.
6. **Bảng điều khiển phân tích (Analytics dashboard):
   - Thống kê các loại câu hỏi phổ biến.
   - Phát hiện các lỗ hổng kiến thức trong KB.

TÀI LIỆU THAM KHẢO

[1] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 10746–10761, doi: 10.18653/v1/2025.findings-emnlp.568.

[2] LlamaIndex, “LlamaIndex Python Documentation,” 2025 . [Online]. Available: https://developers.llamaindex.ai/python/framework/llama_cloud/llama_parse/ [Accessed: Nov. 2025].

[3] Tavily AI, “Tavily Search API documentation,” 2024. [Online]. Available: https://docs.tavily.com/. [Accessed: Nov. 2025].

[4] Neo4j, Inc., “Building knowledge graphs for AI applications,” 2023. [Online]. Available: https://neo4j.com/developer/ai/. [Accessed: Dec. 2025].

[5] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[6] LangChain AI, "LangGraph Documentation," *LangChain*, [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Nov. 2025].

[7] OpenAI, “Building applications with large language models,” 2024. [Online]. Available: https://platform.openai.com/docs. [Accessed: Dec. 2025].

[8] Facebook AI Research, “FAISS documentation,” 2024. [Online]. Available: https://faiss.ai/. [Accessed: Dec. 2025].

[9] LangFuse, "LangFuse: Open Source LLM Engineering Platform," *LangFuse Docs*, [Online]. Available: https://langfuse.com/docs. [Accessed: Nov. 2024].